

# Automated Component Recovery and Custom Kit Assembly System for Educational Circuit Kits

1<sup>st</sup> Aman Mishra  
*Computer Eng. and Informatics*  
*Middlesex University Dubai*  
Dubai, UAE  
a.mishra@mdx.ac.ae

2<sup>nd</sup> Faseeh Mohammed  
*Computer Eng. and Informatics*  
*Middlesex University Dubai*  
Dubai, UAE  
ff416@live.mdx.ac.uk

3<sup>rd</sup> Dr. Judhi Prasetyo  
*Computer Eng. and Informatics*  
*Middlesex University Dubai*  
Dubai, UAE  
j.prasetyo@mdx.ac.ae

**Abstract**—Educational institutions issue circuit development kits, such as Arduino modules, for practical coursework, but returned kits often lack critical components and cannot be redistributed as complete sets. This paper proposes an automated robotic solution to this logistical and environmental inefficiency. Unlike traditional assembly systems that build standard products from uniform supply lines, the proposed system recovers components from existing, incomplete kits. A Python machine-vision system identifies available modules and communicates via TCP/IP with an EPSON RC8.0 simulator controlling a VT6-A901S 6-axis robot, enabling dynamic assembly of new kits from custom Bills of Quantities (BOQs) submitted for specific projects. The result is a flexible, waste-reducing system that gives students exactly the components their modules require.

**Index Terms**—Robotic Assembly, Machine Vision, Component Recovery, EPSON RC8.0, Automation, E-waste.

## I. INTRODUCTION

Practical electronics and robotics education relies heavily on development kits, typically built around microcontrollers such as the Arduino alongside sensors, actuators, and passive components. Although students are encouraged to keep these kits, many return them at the end of term, often with modules missing, damaged, or misplaced. Because the returned kits are incomplete, they cannot be re-issued to incoming students who need standard, comprehensive sets, so institutions accumulate partial kits, incurring unnecessary procurement costs and electronic waste.

This project develops a robotic system that sorts returned components and dynamically rebuilds kits. By targeting project-specific requirements rather than rigid standard kit definitions, the system reallocates whatever components a returned kit contains to the students whose projects actually need them, even when generic parts are absent.

## II. METHODOLOGY

### A. Evaluation of Existing Solutions

Automated kitting and assembly are well established but almost exclusively designed for factory-stage production, typically using Delta and SCARA (Selective Compliance Assembly Robot Arm) robots to pick uniform components from controlled feeder lines. Automated sortation is also used in large-scale waste management: facility-scale smart sortation systems combine real-time machine vision with air jets and Delta arms to recover valuable commodities, including e-waste, from mixed municipal solid waste [1]. These systems prove the efficacy of AI-guided robotic sorting for high-volume recovery, but operate at facility scale for gross sorting rather than the precise, project-specific kitting of delicate microelectronic modules needed in education.

These methodologies share one assumption: a continuous, uniform supply of new components. No widely adopted solution recovers loose, mixed components from partially depleted kits to build newly configured ones. Recent work on e-waste recovery stresses that salvaging used electronics differs fundamentally from original production: recovery is characterized by high uncertainty, as components may be damaged, randomly oriented, or missing [2]. Deterministic factory automation is therefore insufficient for these Re-X (Recycling, Re-use, Re-manufacturing) activities, motivating the flexible, data-driven approach proposed here.

### B. Proposed Unique Approach

The solution's differentiating factor is intelligent recovery and flexible kitting. Rather than having staff manually audit and sort thousands of components to rebuild standard kits, the system operates on a custom Bill of Quantities (BOQ) model: staff or students submit a

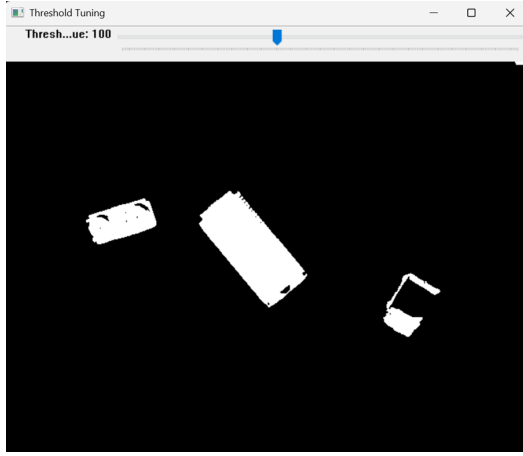


Fig. 1. Image thresholding using openCV

project BOQ, and the system identifies the required parts from the pool of returned components and assembles a kit tailored to that project. This shifts the institution from a “push” model (a standard kit for everyone regardless of need) to a “pull” model (kits assembled strictly from project requirements), maximizing the utility of returned inventory.

### III. IMPLEMENTATION

To realize this automated recovery and assembly system, a simulated robotic workcell has been developed, integrating high-level software control with industrial robotic simulation.

#### A. Machine Vision System

The machine-vision architecture is the core of the pick-and-place operation: it scans the unstructured workbench, identifies components, decodes their orientation, and translates pixel data into Cartesian coordinates for the EPSON VT6-A901S manipulator. Many industrial bin-picking solutions rely on line-of-sight 3D cameras, but these generate point clouds prone to spurious outliers on cluttered, overlapping parts, which degrade obstacle-avoidance planning and segmentation and require costly filtering [3]. To avoid 3D point-cloud filtration and keep real-time processing efficient, this project uses a 2D planar overhead approach. The system developed through two phases, culminating in a deep-learning solution.

1) *Prototype Phase: Deterministic Computer Vision:* The initial architecture relied on deterministic OpenCV processing. Frames from an overhead 720p webcam were Gaussian-blurred to suppress noise, then global binary thresholding (Fig.1) isolated geometric footprints. Contour analysis (`cv2.findContours`) found each object’s boundary, and a scale-preserving script padded the

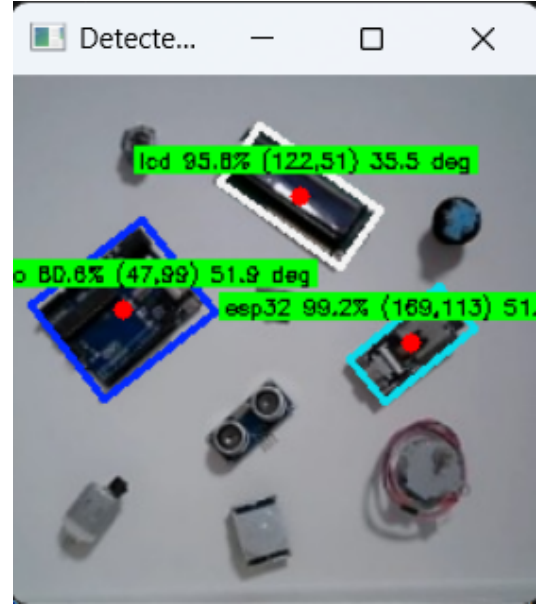


Fig. 2. Image inference using yoloV8

crops into uniform  $224 \times 224$  tensors for a Google Teachable Machine classifier. Although functional in controlled conditions, the prototype suffered severe domain-shift failures: metallic glare erased texture, background clutter produced false “Noise” classifications, and axis-aligned bounding boxes could not derive the resting rotation angle (yaw).

2) *Final Architecture: YOLOv8-OBb (Oriented Bounding Box):* To overcome these limitations, the system was re-architected around a YOLOv8-OBb neural network that ingests the full-resolution, uncropped camera frame, eliminating manual thresholding and cropping. Trained on a custom, heavily augmented Computer Vision Annotation Tool dataset (extreme lighting shifts and rotational variance), the OBb model bypasses glare and localized clutter (Fig.2). The manual annotation this supervised approach requires is a known but necessary trade-off for the recognition quality and spatial accuracy that bin-picking demands [4]. In a single forward pass it outputs a 5-D oriented tensor ( $cx, cy, width, height, theta$ ) with multi-class probabilities (Arduino, ESP32, LCD). This is far simpler than hybrid pipelines that, for example, use YOLOv5 for top-view detection followed by an eye-in-hand FAST/BRISK feature-matching step for pose estimation [5]. By deriving the resting angle ( $theta$ ) from a single overhead frame, the OBb architecture removes any dependence on perfect lighting or multi-step image capture.

3) *Logistics Filtering and BOQ Engine:* Once the vision system extracts each component’s pixel centroid

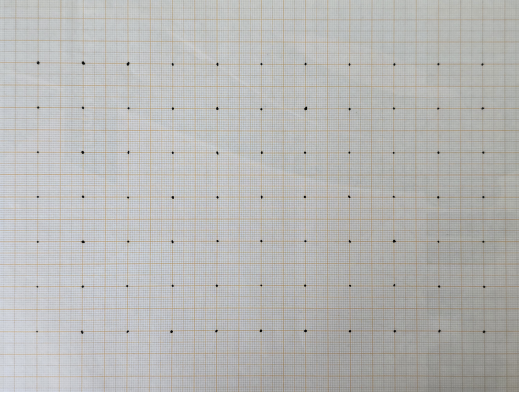


Fig. 3. 77-point homography calibration grid showing pixel click positions overlaid on the workspace reference image.

$(X, Y)$  and class label, a logistics filter operates above the frame buffer. Rather than greedily forwarding every detection to the coordinate-mapping pipeline, the orchestrator matches the detected inventory against a Bill of Quantities (BOQ), ignoring surplus components even when they are validly detected, so that only the required, correctly kitted items are passed to the spatial calibration engine for EPSON translation.

### B. Spatial Calibration and Coordinate Mapping

A critical requirement is translating pixel positions in the camera image into physical millimetre coordinates in the robot’s world frame, achieved through a planar homography that provides a projective mapping between the two coordinate spaces.

The mapping uses a 77-point calibration grid of 11 columns by 7 rows at 20 mm spacing in both axes (Fig.3). The world X-coordinate decreases from +100 mm to −100 mm left to right and the world Y-coordinate increases from 410 mm to 530 mm top to bottom, giving a calibrated area of 200 mm × 120 mm. A reference image of the grid (`graph_paper.jpg`) is captured with the camera normal to the workspace; `tools/calibration_clicker.py` records the 77 intersections, and `tools/sort_and_tag_pixels.py` sorts them column by column and appends the world coordinates to produce a four-column pixel-to-world CSV.

The homography matrix  $\mathbf{H}$  is computed with OpenCV’s `findHomography` using standard least squares (method 0), not RANSAC. This is deliberate: RANSAC rejects outliers, but the corner points define the workspace extremes and are valid, important constraints, so treating them as outliers would introduce systematic error near the boundaries. With 77 evenly distributed points, least squares gives a robust estimate

TABLE I  
TCP/IP COMMAND SET

| Command | Parameters | Robot Action                                                      | Reply |
|---------|------------|-------------------------------------------------------------------|-------|
| JUMP    | x, y, z, u | Lifts 50 mm, moves to $(x, y, z+50)$ , descends to $(x, y, z, u)$ | OK    |
| GO      | x, y, z    | Linear motion to $(x, y, z)$                                      | OK    |
| MOVE    | x, y, z    | Linear motion to $(x, y, z)$                                      | OK    |
| PICK    | x, y, z, u | Full pick-and-place sequence (Section III-D)                      | OK    |
| STANDBY | —          | Returns to home position                                          | OK    |

without outlier rejection. At runtime, any detected centroid  $(p_x, p_y)$  is mapped to world coordinates  $(w_x, w_y)$  via `cv2.perspectiveTransform` with  $\mathbf{H}$ .

Repositioning the camera vertically changes the image scale and invalidates the pixel-to-millimetre mapping. To avoid a full recalibration, a height-recalibration routine (`pickerbot.py --calibrate`) lets the user click two points known to be exactly 20 mm apart. Let  $r_h$  and  $r_v$  denote the mean pixel distance per 20 mm in the horizontal and vertical axes from the original calibration. For a measurement at angle  $\theta$  to the horizontal, the expected pixel distance is the anisotropic norm:

$$d_{\text{expected}} = \sqrt{(r_h \cos \theta)^2 + (r_v \sin \theta)^2} \quad (1)$$

where  $\theta = \arctan 2(|\Delta y|, |\Delta x|)$ . The scale factor is then:

$$s = \frac{d_{\text{measured}}}{d_{\text{expected}}} \quad (2)$$

This accounts for horizontal and vertical pixel densities differing due to lens geometry or non-square sampling. All 77 coordinates are scaled about the grid centre (index 38) by  $s$ , written to `calibration_pixels_scaled.csv`, and referenced in `config.json` for subsequent runs.

### C. TCP/IP Communication Protocol

Table I summarises the full command set supported by the system.

The Python control system communicates with the EPSON RC8 controller over a local TCP socket. For simulation the target is 127.0.0.1 on port 2001; for the physical robot the address is changed to 192.168.X.Y via `config.json`. Connection management and dispatch are handled by `pickerbot_lib.sender`, which holds a module-level socket for the session.

Commands are plain ASCII strings of the form `COMMAND X Y Z U\r\n`, where  $X, Y, Z$  are world coordinates in millimetres and  $U$  is the tool rotation in

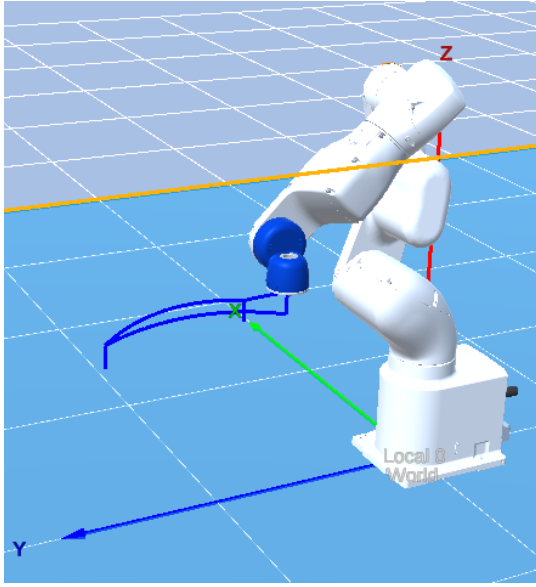


Fig. 4. Trajectory of the PICK command: three-phase Jump3 approach to the component, gripper activation, transport to PlacePoint, and release.

degrees. The controller replies "OK" after each command, and a small inter-command delay allows mechanical completion before the next is sent.

For multi-component picks, `epsonPickAll` iterates over a list of world-coordinate strings and issues a PICK for each, checking the reply against "OK". Any non-OK response aborts the batch immediately and returns the robot to standby, so a partial failure such as a grip error or controller fault cannot leave the robot operating in an undefined state.

Fig.4 shows a sample pick-and-place: from standby, the arm approaches the object in the correct orientation, picks it, jumps to the place point, fixes orientation, places, and retracts to standby for the next operation.

#### D. EPSON RC8 Robot Program

The robot-side program (`Epson/Pickerbot_Receiver/Main.prg`) is written in Epson SPEL+, the RC8 controller's scripting language, building on a template script [6] with predefined motion-initialization parameters. After initialization it opens a TCP server on channel #201 at port 2001 and waits for a client before entering its command loop.

Each string is split on spaces into the command token and floating-point coordinate arguments. The PICK command is the most complex: a three-phase Jump3 trajectory lifts the arm vertically, translates to an elevated waypoint above  $(x, y)$  with the tool at orientation  $u$ , and descends to the pick position, where a digital output

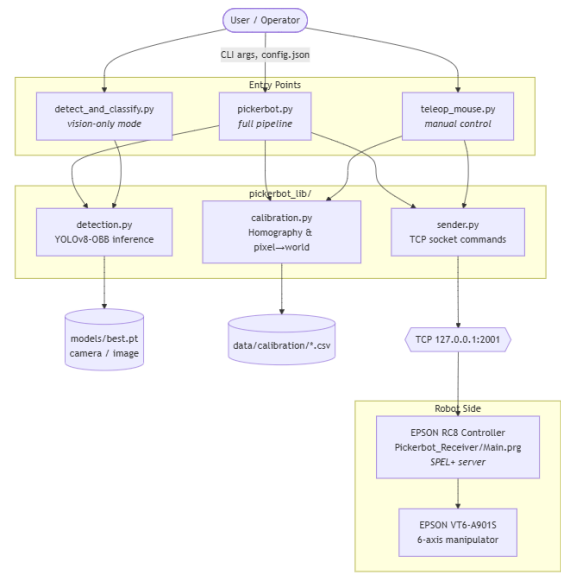


Fig. 5. Software architecture of the Picker-Bot system, illustrating the three entry-point scripts and their shared dependency on the pickerbot\_lib package.

closes the gripper and a dwell ensures a reliable grip. A second Jump3 lifts clear and transports the component to a predefined PlacePoint (the target kit container), where the output is deactivated to release it before the arm returns to standby.

A network error handler reopens the server socket and resumes listening rather than halting, letting the system recover from transient network interruptions without operator intervention.

#### E. Software Architecture and Entry Points

The codebase (Fig.5) is built around a shared package, `pickerbot_lib`, consolidating detection, calibration, and TCP communication into three submodules: `detection`, `calibration`, and `sender`. Three entry-point scripts at the project root each import this package to serve a distinct mode.

The primary entry point, `pickerbot.py`, orchestrates an end-to-end pick session. A `--src` argument selects the detection source (live camera or a static image), and `--calibrate` launches the height-recalibration GUI and exits. In a normal run it loads the homography CSV named in `config.json`, runs detection to obtain centroid pixels, maps each to world coordinates, and dispatches the list to `epsonPickAll` over TCP.

The second, `detect_and_classify.py`, isolates the vision layer for development and testing without a robot connection: given a file path it processes a static image, and without arguments it opens the live feed with a real-time confidence-threshold slider.

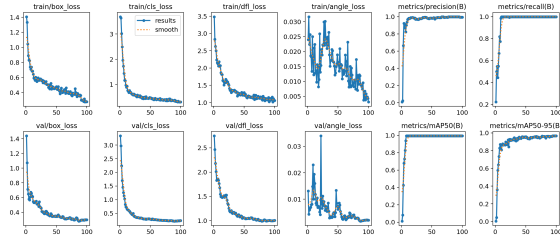


Fig. 6. YOLOv8-OBb training metrics. Loss curves and  $mAP_{50}$  progression demonstrating rapid convergence and high detection accuracy on the in-distribution component set.

The third, `teleop_mouse.py`, provides interactive positioning: it loads the calibration, connects to the robot, and displays the reference image with all 77 points annotated. Each click is transformed through  $\mathbf{H}$  and sent as a MOVE command, enabling manual positioning without the full pipeline, which is useful for verifying calibration accuracy before an automated session.

#### F. Robotic Control and Simulation

Manipulation is simulated in the EPSON RC8.0 environment. The system can cover any workspace size; the initial setup mapped a  $200 \text{ mm} \times 120 \text{ mm}$  evaluation space with a deposit point  $280 \text{ mm}$  above its centre. The platform is the EPSON VT6-A901S [7], a 6-axis arm with  $980 \text{ mm}$  reach suited to pick-and-place requiring varied approach angles and high repeatability; full installation, safety, and operational specifications appear in the Epson VT6 documentation [8].

### IV. RESULTS

The Picker-Bot system integrated deep-learning vision with industrial robotic control inside the EPSON RC8.0 simulator. The following subsections report observed behaviour across three layers: perception, spatial reasoning, and supervised manipulation.

#### A. Vision System Performance and Robustness

The migration from the OpenCV prototype to the YOLOv8-OBb architecture substantially improved detection reliability. The final model classified Arduino, ESP32, and LCD modules and recovered the resting angle  $\theta$  with the centroid  $(cx, cy)$  in a single forward pass, with no geometric post-processing, and absorbed the glare, clutter, and background texture that had destabilised the prototype. An explicitly trained “noise” class was suppressed at the output, so only valid components reached the coordinate-mapping layer.

Fig. 6 plots the training loss and mean Average Precision: the model converged rapidly to a final  $mAP_{50}$

of 0.99, indicating near-perfect localisation and classification within the training distribution. A fixed confidence threshold of 0.7 eliminated the few remaining glare-induced false positives, with no additional non-maximum suppression required.

#### B. Spatial Calibration and Mapping Accuracy

The homography reliably bridged the camera image and the planar workspace, with the 77-point grid defining a calibrated region of roughly  $200 \text{ mm} \times 120 \text{ mm}$  that covered the full overhead field of view. Using least squares rather than RANSAC kept the boundary corner points from being discarded as outliers, which would otherwise have contracted the calibrated area.

Height recalibration absorbed camera repositioning in seconds without re-clicking every intersection: measuring a known  $20 \text{ mm}$  segment against the original grid pitches recovered an anisotropic scale factor applied about the grid centre. The result was verified through `teleop_mouse.py`, where each click produced a MOVE command whose terminal position visually matched the clicked feature.

#### C. Integrated Robot Control and Logistics

The control pipeline exercised the pull-based kitting end-to-end. Given a BOQ, the system satisfied its requirements in order of detection confidence and ignored correctly recognised but unneeded components, behaving as a logistics auditor rather than a greedy collector; without a BOQ it reverted to greedily picking every non-noise detection.

The TCP/IP interface dispatched the full command set (JUMP, GO, MOVE, PICK, STANDBY) reliably across extended runs, each validated by an OK reply before the next was sent. The PICK primitive executed its three-phase Jump3 trajectory in every trial, and synthetic failures exercised the abort path, returning the manipulator to standby rather than continuing through an undefined state. Across all configurations the system switched cleanly between live USB and pre-recorded images via a single `config.json` setting, interfaced with the RC8.0 simulator at `127.0.0.1`, and kept 0.7 as the default confidence threshold while exposing a runtime slider.

### V. FUTURE DISCUSSION

The current implementation demonstrates the concept in simulation; several avenues remain for further development.

The most immediate step is deployment to physical hardware. Moving from the RC8.0 simulator to a real

VT6-A901S will require calibrating the physical camera under varying lighting and tuning gripper timing to the mechanical characteristics of real components; handling delicate modules of varying shape and mass may also require an adaptive or compliant gripper to avoid damage.

Further ahead, a web-based portal would let students and staff submit BOQs directly; the system could queue requests, process inventory overnight, and notify users when a kit is ready, making recovery self-service and reducing staff workload. To automate the physical logistics of the pull model, future iterations could integrate Internet of Things (IoT) technologies and Autonomous Guided Vehicles (AGVs), increasingly used in flexible manufacturing to route materials across facilities [9]: an IoT-guided AGV network could fetch bins of returned kits to the workcell and deliver finished kits to student collection points.

## VI. CONCLUSION AND DISCUSSION

### A. Critical Evaluation of System Integration

The Picker-Bot system successfully shifted component kitting from a “push” model to a project-specific “pull” model. Its technical cornerstone was the migration from deterministic OpenCV prototypes to YOLOv8-ORB: where the prototype was highly sensitive to environmental shifts, the final model reached a  $mAP_{50}$  of 0.99, showing that high-accuracy bin-picking is viable in cluttered educational settings without 3D point-cloud hardware. The “logistics auditor” approach—filtering detections through the BOQ engine—lets the system both detect parts and manage inventory, directly addressing the goal of reducing procurement costs and e-waste.

### B. Implementation Constraints and Real-World Considerations

Several constraints would affect any physical deployment. The fixed software delay after each OK kept the simulated pipeline stable but should instead be tuned to each component’s inertia, since a heavier LCD settles differently from a lighter ESP32 and a single value trades throughput against reliability. The 77-point homography is a planar model of a three-dimensional problem: components of non-negligible height introduce a parallax offset that appears as a small systematic lateral error during the Jump3 descent, so a height-aware correction (per-class offsets or a depth sensor) would be needed. Finally, the abort-on-failure strategy—correct as an industrial default—is blunt at higher throughputs, where a graduated model distinguishing transient communication

faults from genuine mechanical errors would be necessary to scale to production volumes.

### C. Concluding Remarks

In conclusion, the Picker-Bot system provides a technically sound, sustainable solution for managing returned educational circuit kits. By integrating Oriented Bounding Box detection, planar homography, and TCP/IP control, it shows that modern AI can turn “loose waste” into project-ready assets, setting a precedent for “Re-X” (Recovery and Re-use) activities in microelectronic environments and bridging waste management with logistical efficiency.

## ACKNOWLEDGMENT

The authors acknowledge the guidance and resources of the PDE4435 module, which encouraged a design-thinking approach to advanced robotics projects, and the use of Claude during project development.

## REFERENCES

- [1] AMP, “AMP ONE: Automated Waste Sorting Systems & Facility,” 2026. [Online]. Available: <https://ampsortation.com/products/amp-one>.
- [2] J. Saenz *et al.*, “Automated disassembly of e-waste—requirements on modeling of processes and product states,” *Frontiers in Robotics and AI*, vol. 11, p. 1303279, 2024.
- [3] M. Franaszek, P. Rachakonda, and K. S. Saidi, “Filtering Organized 3D Point Clouds for Bin Picking Applications,” *Applied Sciences*, vol. 14, no. 3, p. 961, Jan. 2024.
- [4] P. Fager, R. Hanson, Å. Fasth-Berglund, and S. Ekered, “Supervised and unsupervised learning in vision-guided robotic bin picking applications for mixed-model assembly,” *Procedia CIRP*, vol. 104, pp. 1304–1309, 2021.
- [5] T. Taweessontorn, S. Yanyong, and P. Konghuayrob, “Hybrid Deep Learning and FAST-BRISK 3D Object Detection Technique for Bin-picking Application,” *Sensors and Materials*, vol. 36, no. 4, pp. 1389–1404, 2024.
- [6] J. Prasetyo, “RC7\_Python,” Jan. 03, 2024. Available: [https://github.com/judhi/RC7\\_Python](https://github.com/judhi/RC7_Python).
- [7] EPSON Robotics, “RC8.0 Reference Manual,” EPSON Factory Automation, 2023. [Online]. Available: <https://www.epson.com>
- [8] Epson America, Inc., “Epson VT6L All-in-One 6-Axis Robots — Support,” Epson US, 2026. [Online]. Available: [https://epson.com/Support/Robots/6-Axis-Robots/VT-Series/Epson-VT6L-All-in-One-6-Axis-Robots/s/SPT\\_VT6L-A901SS](https://epson.com/Support/Robots/6-Axis-Robots/VT-Series/Epson-VT6L-All-in-One-6-Axis-Robots/s/SPT_VT6L-A901SS).
- [9] I. Vlachos, R. M. Pascazzi, M. Ntosis, K. Spanaki, S. Despoudi, and P. Repoussis, “Smart and flexible manufacturing systems using Autonomous Guided Vehicles (AGVs) and the Internet of Things (IoT),” *International Journal of Production Research*, pp. 1–22, 2022.